

CounselConnect

Complete Project Documentation

Complete Project Documentation

AI-Assisted Counseling Management Platform

Major Project · Team ID: P7_056

Department of Computer Engineering

Faculty of Engineering & Technology

Farhan Sargath · Nakul Dabhi · Kavya Vaghela

Internal Guide: Prof. Priyanka Mangi

Generated from the CounselConnect source code

CounselConnect — Complete Project Documentation

AI-Assisted Counseling Management Platform Major Project · Department of Computer Engineering
Team ID: P7_056 · Farhan Sargath, Nakul Dabhi, Kavya Vaghela

1. System Overview

CounselConnect is a full-stack web platform connecting people seeking mental health support with professional counselors. It is a **single React application** serving three distinct role-based panels, backed by a **Node.js / Express REST API** with a **MongoDB** database and a **Socket.IO** real-time channel.

Scale

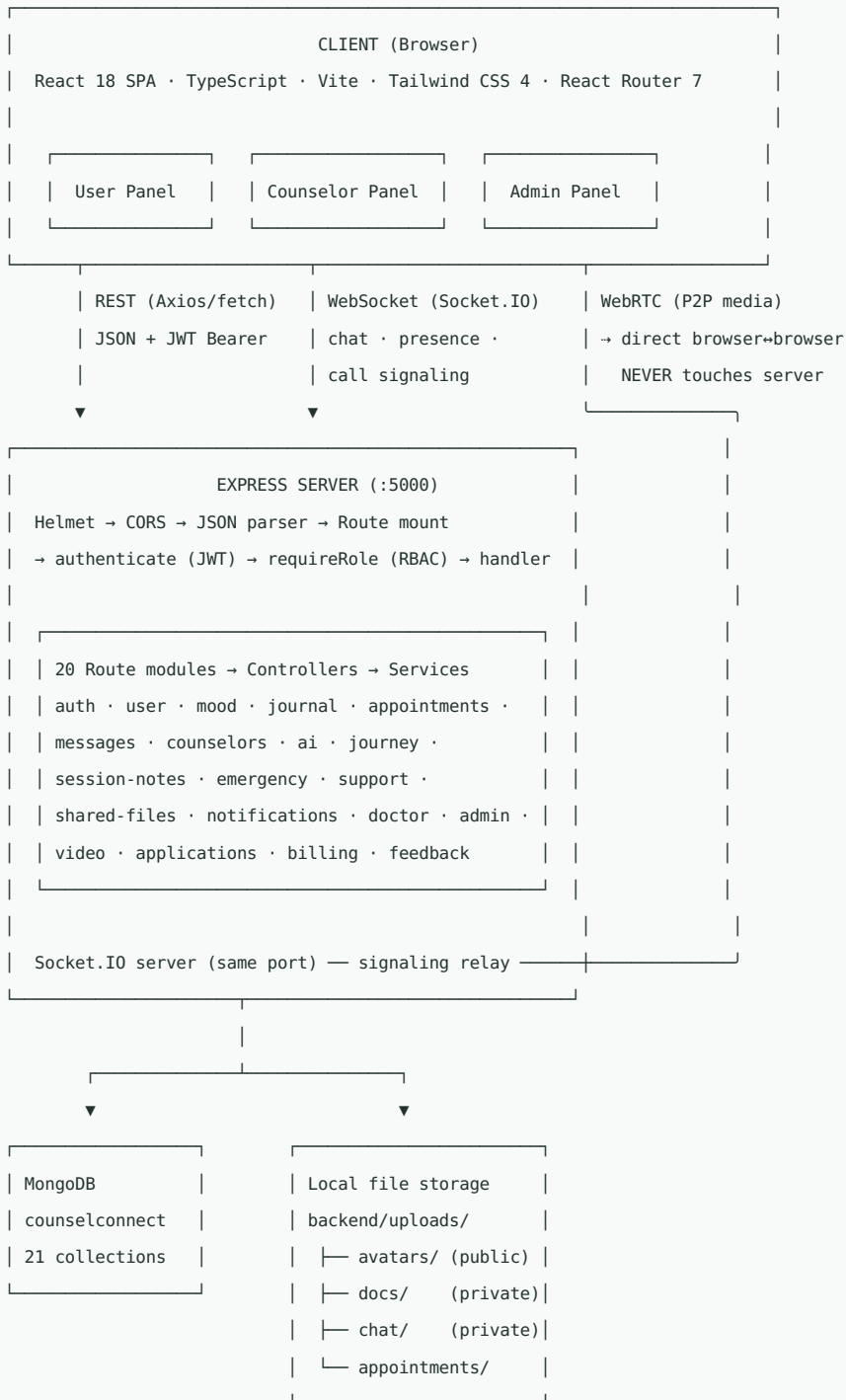
Metric	Value
Backend JavaScript	~11,000 lines across 85 files
Frontend TypeScript/React	82 application files + 48 shadcn UI components
REST API endpoints	180 across 20 route modules
MongoDB collections	21
Socket.IO events	14
PDF generators	7
User roles	3 (user, doctor, admin)

The three panels

Panel	Route	Who
User (Client)	<code>/dashboard/*</code>	People seeking counseling
Counselor (Doctor)	<code>/doctor</code>	Verified counselors
Admin	<code>/admin</code>	Platform administrators

All three sign in at the same page (`/login`). The server issues a JWT containing the role; the front-end routes to the correct panel and route guards prevent cross-panel access.

2. Architecture



Request lifecycle

Browser

```
→ fetch('/api/appointments', { Authorization: 'Bearer <jwt>' })
→ Express: helmet → cors → express.json()
→ Router: /api/appointments
→ authenticate middleware — verifies JWT signature, rejects 2FA-stage tokens
→ requireRole('user')    — RBAC check
→ express-validator       — input validation
→ controller              — HTTP concerns only
→ service                 — business logic (no Express objects)
→ fileStore.utils         — readStore('appointments.json')
→ mongoStore.utils        — in-memory cache, loaded from MongoDB at boot
← JSON { success, message, data }
```

3. Complete File Structure

```

Major Project Final/
|
├─ backend/                                Node.js + Express API
|   ├─ server.js                          Entry point – connects DB, then listens
|   ├─ app.js                             Express app, middleware, route mounting
|   ├─ seed.js                            Creates demo accounts for all 3 roles
|   ├─ package.json                       Backend dependencies + npm scripts
|   ├─ .env                               PORT, JWT_SECRET, MONGODB_URI, ...
|   |
|   └─ config/
|       ├─ app.config.js                  Port, JWT secret/expiry, upload limits
|       ├─ cors.config.js                 Allowed origins
|       └─ db.config.js                   MongoDB URI, database name, timeout
|   |
|   └─ middleware/
|       ├─ auth.middleware.js             JWT verification + requireRole (RBAC)
|       ├─ error.middleware.js            Centralised error → JSON envelope
|       └─ validate.middleware.js         express-validator result handler
|   |
|   ├─ routes/                           (20 files) URL → controller mapping
|   ├─ controllers/                       (20 files) HTTP request/response handling
|   └─ services/                          (25 files) Business logic
|   |
|   └─ utils/
|       ├─ fileStore.utils.js             Storage API – delegates to MongoDB
|       ├─ mongoStore.utils.js            MongoDB engine (cache + write-through)
|       ├─ jwt.utils.js                   Sign/verify/blacklist tokens
|       ├─ password.utils.js              bcrypt hash + compare (12 salt rounds)
|       ├─ totp.utils.js                  RFC 6238 two-factor authentication
|       └─ response.utils.js              success()/error() JSON envelope
|   |
|   └─ realtime/
|       └─ signaling.js                   Socket.IO – chat, presence, WebRTC signaling
|   |
|   └─ scripts/
|       ├─ db-status.js                   npm run db:status
|       ├─ db-import.js                   npm run db:import
|       └─ db-export.js                   npm run db:export (run before zipping)
|   |
|   └─ data/                             (21 files) JSON snapshot / MongoDB fallback + transfer
|       └─ uploads/                       Uploaded files (avatars, docs, chat, intake)
|   |
├─ src/                                   React frontend
|   ├─ main.tsx                           React root
|   └─ app/
|       ├─ Root.tsx                       App shell
|       ├─ App.tsx                         Public site (landing, login, register)
|       ├─ routes.ts                       Route table
|       ├─ DoctorPanel.tsx                Counselor panel shell + routing
|       |
|       └─ context/AuthContext.tsx         Auth state provider
|   |

```

```

|   └─ lib/
|   |   └─ api.ts           Central API client (JWT, downloads, uploads)
|   |   └─ auth.ts         Token/user storage helpers
|   |   └─ callClient.ts    WebRTC engine (CallSession class)
|   |   └─ callInbox.ts     Ringer → video page hand-off
|   |   └─ chatClient.ts    Socket.IO chat channel
|   |   └─ useMediaStream.ts Binds MediaStream to <video>
|   |   └─ colors.ts        User panel design tokens
|   |
|   └─ components/
|   |   └─ DashboardLayout.tsx User panel shell + sidebar
|   |   └─ Navbar.tsx         Public navigation
|   |   └─ IncomingCallRinger.tsx Global incoming-call UI
|   |   └─ ScrollToTop.tsx
|   |   |
|   |   └─ pages/ (22 files) User panel + public pages
|   |   └─ doctor/ (21 files) Counselor panel pages
|   |
|   └─ admin/
|   |   └─ App.tsx           Admin shell + routing
|   |   └─ components/ (19) Admin pages
|   |   └─ context/ThemeContext.tsx Light/dark theming
|   |   └─ lib/adminApi.ts   Admin API client
|
└─ package.json             Frontend dependencies
└─ vite.config.ts           Build configuration
|
└─ Documentation/
    └─ PROJECT-DOCUMENTATION.md ← this file
    └─ TECH-STACK-AND-ALGORITHMS.md Libraries, algorithms, justifications
    └─ DATABASE.md           MongoDB architecture
    └─ INSTALL-MONGODB.md    MongoDB setup walkthrough
    └─ HOW-TO-RUN.md         Setup guide for teammates
    └─ RUN-PROJECT.md        Daily run instructions
    └─ BACKLOG.md            Done / not-done register
    └─ start-project.bat     One-click launcher

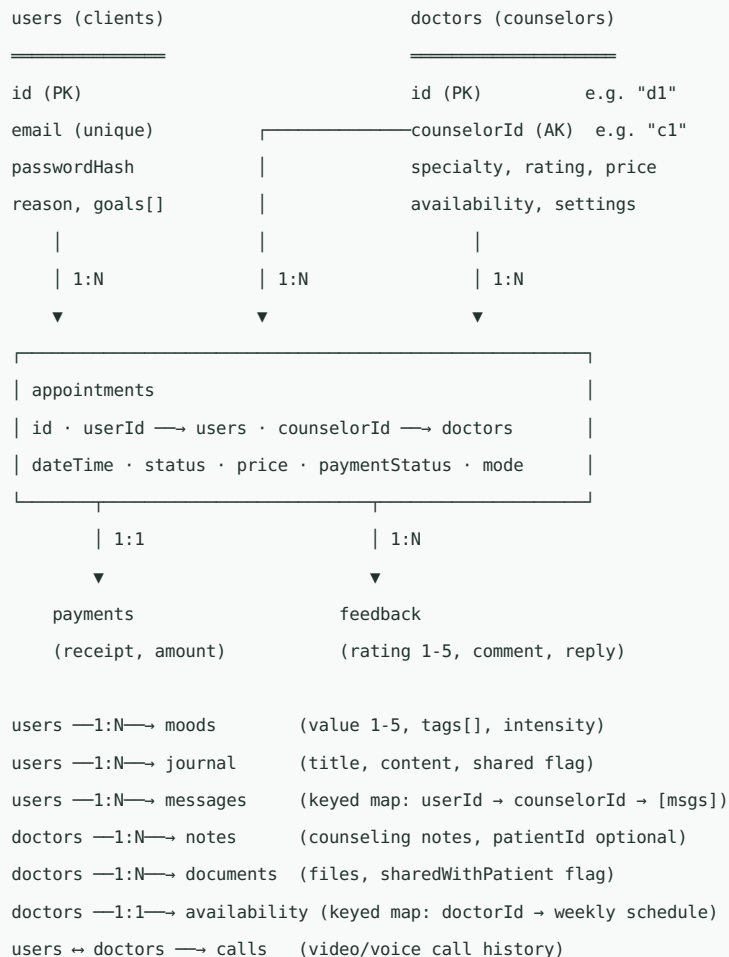
```

4. Database Schema

Database: `counselconnect` (MongoDB) · **21 collections**

Each document keeps its own `id` field and reuses it as MongoDB's `_id`, so collections are readable in Compass and reference lookups stay natural.

4.1 Entity relationships



4.2 Collection reference

users — client accounts

Field	Type	Notes
id	String (UUID)	Primary key
firstName, lastName	String	
email	String	Unique — login identifier
passwordHash	String	bcrypt, 12 salt rounds
phone, bio, avatar	String	Profile
reason	String	Presenting concern — drives AI matching
sessionType	String	Video / Chat / Mix
goals	Array\	Therapy goals
notifications	Object	{sessions, moodReminders, messages, ...}
privacy	Object	{shareProgress, anonymousData, profileVisible}
status	String	Active / Suspended
createdAt, updatedAt	ISO String	

doctors — counselor accounts

Field	Type	Notes
id	String	"d1" — account key
counselorId	String	"c1" — public-facing id used in all relations
name, firstName, lastName	String	
email	String	Unique
passwordHash	String	bcrypt
title, specialty, approach	String	Professional profile
rating	Number	Recomputed from feedback
sessions	Number	Lifetime count
experience, languages[]		
available	Boolean	Accepting new clients
price	Number	Per-session fee
status	String	Verified / Pending
settings	Object	Notification prefs, darkMode, 2FA

Note the dual key. A counselor has both `id` (`d1`) and `counselorId` (`c1`). Auth and doctor-panel routes use `id` ; every client-facing relation (appointments, feedback, messages) uses `counselorId` . Be ready to explain this in viva — it separates the account from the public profile.

appointments

Field	Type	Notes
id	String (UUID)	PK
userId	String	→ users.id
counselorId	String	→ doctors.counselorId
counselorName , counselorAvatar	String	Denormalised for fast list rendering
sessionType	String	video / chat
mode	String	online / offline
date , time	String	Display strings
dateTime	ISO String	Canonical sort/compare value
price	Number	
reason	String	What the client wants to work on
documents	Array	Client-attached intake files
status	Enum	pending → confirmed → completed , or rejected / cancelled
paymentStatus	Enum	unpaid / paid / refunded
outsideHours	String null	Set when booked outside counselor hours
acceptedAt , rejectionReason		

moods

Field	Type	Notes
id , userId	String	
value	Number 1-5	Core mood score
intensity	Number 1-10	Optional
label , emoji	String	"Good", :)
tags	Array\	Context — work, sleep, money... drives recommendations
notes	String	
date , createdAt		

payments

Field	Type	Notes
id , receiptNumber	String	CC-2026-45X68Q
userId , appointmentId , counselorId	String	
amount , currency		
platformFee	Number	20%
counselorPayout	Number	80%
method , methodLabel	String	card / upi / netbanking
gatewayReference	String	SIM-...
simulated	Boolean	true — no real gateway
status	Enum	paid / refunded

Remaining collections

Collection	Shape	Purpose
admins	Array	Admin accounts
applications	Array	Counselor applications awaiting review
availability	Map doctorId →	Weekly schedule + vacation/break/auto-reject settings
calls	Array	Video/voice call history with duration
crisis-log	Array	Emergency-page usage (anonymised)
documents	Array	Counselor files, sharedWithPatient flag
feedback	Array	Ratings 1-5, comment, counselor reply
journal	Array	Client journal entries, shared flag
logins	Array	Login history (device, IP, timestamp) — capped at 200
messages	Map userId → counselorId → []	Chat threads
notes	Array	Counseling notes, optional patientId / appointmentId
notification-reads	Map doctorId → [ids]	Doctor read state
notifications	Array	Per-user notifications
notifications-read	Array	User read state
platform-notifications	Array	Admin broadcasts (with scheduledFor)
settings	Map	Platform + per-account settings

4.3 Design decisions to defend

Deliberate denormalisation. appointments stores counselorName and counselorAvatar alongside counselorId . Rendering a booking list therefore needs no second lookup. This is a standard NoSQL trade — read speed over strict normalisation — and it is the correct answer when an examiner asks "is your data normalised?"

Map-shaped collections. Four collections are key/value maps rather than lists because they are always accessed by owner (a user's threads, a doctor's schedule). In MongoDB the map key becomes `_id`.

Recommended indexes (state these in viva): `users.email` (unique), `doctors.email` (unique), `doctors.counselorId`, `appointments.userId`, `appointments.counselorId`, `appointments.dateTime`, `moods.userId`, `feedback.counselorId`.

5. Backend — File by File

Entry & configuration

File	Responsibility
<code>server.js</code>	Connects to MongoDB and loads all collections <i>before</i> <code>app.listen()</code> — services read synchronously, so listening early would serve requests against an empty cache. Attaches Socket.IO. Flushes pending writes on SIGINT/SIGTERM.
<code>app.js</code>	Express instance. Helmet → CORS → JSON parser → static uploads → 20 route mounts → error middleware. Also serves <code>/api/health</code> reporting DB engine + document count.
<code>config/ app.config.js</code>	Port, JWT secret and expiry (7d), upload limits
<code>config/ db.config.js</code>	MongoDB URI, database name, connect timeout, <code>MONGODB_REQUIRED</code>
<code>config/ cors.config.js</code>	Allowed frontend origins

Middleware

File	Responsibility
<code>auth.middleware.js</code>	<code>authenticate</code> verifies the JWT and attaches <code>req.user</code> . Rejects tokens with stage: '2fa' — a challenge token proves only the password was right, so it must never open the API. <code>requireRole(...)</code> enforces RBAC.
<code>validate.middleware.js</code>	Collects express-validator errors into one 400 response
<code>error.middleware.js</code>	Converts thrown errors (with <code>statusCode</code>) into <code>{success:false, message}</code>

Utilities

File	Responsibility
<code>fileStore.utils.js</code>	The storage API every service calls: <code>readStore</code> , <code>writeStore</code> , <code>readStoreObj</code> , <code>writeStoreObj</code> . Delegates to MongoDB when connected, falls back to JSON files when not. Declares the 21 stores.
<code>mongoStore.utils.js</code>	MongoDB engine. Loads collections into memory at boot, serves reads synchronously, writes through on an ordered queue. Shapes documents so they read well in Compass.
<code>jwt.utils.js</code>	<code>signToken</code> , <code>verifyToken</code> , <code>blacklistToken</code> (in-memory logout set)
<code>password.utils.js</code>	bcrypt hash/compare, 12 salt rounds
<code>totp.utils.js</code>	RFC 6238 TOTP: base32 codec, HMAC-SHA1 HOTP, dynamic truncation, ± 1 step window, <code>timingSafeEqual</code> comparison
<code>response.utils.js</code>	<code>success(res, data, message, code)</code> / <code>error(...)</code> — one envelope everywhere

Services (business logic)

Service	Lines	Responsibility
<code>doctor.service.js</code>	1599	Counselor panel: dashboard, patients, appointments, notes, documents, analytics, reports, AI assistant, availability
<code>admin.service.js</code>	1158	Admin: dashboard, users, counselors, appointments, payments, notifications, reports, audit log, settings
<code>pdf.service.js</code>	894	Seven PDF generators via PDFKit
<code>ai.service.js</code>	395	Counselor matching, mood insights, journey summary, recommendations
<code>auth.service.js</code>	383	Register, login, 2FA, password change, login history
<code>applications.service.js</code>	294	Counselor application → admin approval → account creation
<code>mood.service.js</code>	253	Mood CRUD, streaks, stats, weekly/monthly reports
<code>appointments.service.js</code>	240	Booking, rescheduling, slot conflict checks, availability enforcement
<code>video.service.js</code>	227	Call records, contacts, history, statistics
<code>billing.service.js</code>	198	Simulated checkout, receipts, refunds
<code>counselors.service.js</code>	196	Public counselor list, filters, sorting, bookable slot generation
<code>passwordReset.service.js</code>	184	OTP reset with expiry, attempt limits, no account enumeration
<code>emergency.service.js</code>	177	Region-aware crisis helplines (public), grounding exercises
<code>support.service.js</code>	175	FAQ, support tickets with admin replies
<code>feedback.service.js</code>	151	Ratings, review submission, counselor rating recomputation
<code>availability.service.js</code>	141	Single authority for when a counselor is bookable
<code>messages.service.js</code>	131	Chat threads, attachments, read receipts
<code>journal.service.js</code>	120	Journal CRUD, sharing with counselor
<code>relationship.service.js</code>	113	Who may contact whom — the core access guard
<code>sessionNotes.service.js</code>	—	Private per-session notes
<code>sharedFiles.service.js</code>	—	Files a counselor shared with a client
<code>journey.service.js</code>	—	Progress timeline

Real-time

`realtime/signaling.js` — Socket.IO server sharing the HTTP port. Authenticates via JWT handshake, places each account in a room (`user:<id>` / `doctor:<id>`), and handles:

Event	Direction	Purpose
presence:list / presence:update	↔	Who is online
chat:message	↔	Instant message delivery
chat:typing / chat:read	↔	Typing indicator, read receipts
call:invite / call:incoming	→	Ring the callee
call:accept / call:accepted	→	Callee answered
call:reject / call:cancel / call:end	→	Termination
webrtc:offer / webrtc:answer / webrtc:ice	↔	Signaling relay only — no media
call:state	↔	Mute / camera-off / screen-share state

6. Frontend — File by File

Core library

File	Responsibility
<code>lib/api.ts</code>	Central API client. Injects the JWT, parses errors safely (surfaces HTTP status on non-JSON responses), handles authenticated file downloads via blob, and deletes the JSON content-type for FormData so multipart uploads work
<code>lib/auth.ts</code>	Token and user persistence, <code>isLoggedIn()</code> , <code>getUser()</code>
<code>lib/callClient.ts</code>	CallSession class — the WebRTC engine. Media capture, peer connection, offer/answer, ICE, retry, reconnect recovery, ring timeout, screen share, device switching, quality stats
<code>lib/chatClient.ts</code>	<code>ChatChannel</code> — Socket.IO messaging with REST fallback
<code>lib/callInbox.ts</code>	Publish/subscribe hand-off from the global ringer to the video page
<code>lib/useMediaStream.ts</code>	Callback-ref hook binding a <code>MediaStream</code> to a <code><video></code> with explicit <code>play()</code>

User panel pages (`components/pages/`)

`LandingPage` · `LoginPage` · `RegisterPage` · `AboutPage` · `JoinAsCounselorPage` · `DashboardHomePage` · `FindCounselorPage` · `AIMatchingPage` · `AppointmentsPage` · `VideoPage` · `ChatPage` · `MoodTrackerPage` · `JourneyPage` · `AISummaryPage` · `BillingPage` · `FeedbackPage` · `MyFilesPage` · `ResourcesPage` · `EmergencyPage` · `SupportPage` · `SettingsPage`

Counselor panel pages (`components/doctor/`)

`DashboardPage` · `AppointmentsPage` · `PatientsPage` · `VideoSessionPage` · `ChatPage` · `AIAssistantPage` · `MoodJourneyPage` · `PatientJournalsPage` · `CounselingNotesPage` · `ReportsPage` · `AnalyticsPage` · `AvailabilityPage` · `FeedbackPage` · `DocumentsPage` · `NotificationsPage` · `SettingsPage` · `ProfilePage` · `SecurityPage` · `HelpPage` — plus `Sidebar`, `TopNav`, `ThemeContext`

Admin panel pages (`admin/components/`)

`Dashboard` · `Users` · `Counselors` · `Applications` · `Appointments` · `Sessions` · `Feedback` · `Payments` · `Analytics` · `Reports` · `Notifications` · `Settings` · `Profile` — plus `Sidebar`, `TopNav`, `ErrorBoundary`

7. Complete API Reference

180 endpoints across 20 modules. All return `{ success: boolean, message: string, data: object }`.

`/api/auth` — Authentication (12)

POST	<code>/register</code>	Create account
POST	<code>/login</code>	Authenticate → JWT (or 2FA challenge token)
POST	<code>/logout</code>	Blacklist token
GET	<code>/me</code>	Current user
POST	<code>/forgot-password</code>	Request OTP
POST	<code>/verify-reset-code</code>	Verify OTP
POST	<code>/reset-password</code>	Set new password
POST	<code>/2fa/verify</code>	Complete 2FA challenge → full JWT
GET	<code>/2fa</code>	2FA status
POST	<code>/2fa/setup</code>	Generate secret + QR
POST	<code>/2fa/confirm</code>	Confirm enrolment → recovery codes
POST	<code>/2fa/disable</code>	Disable (password-confirmed)

`/api/user` — Profile (7)

GET/PUT	<code>/profile</code> · <code>POST /profile/photo</code>
PUT	<code>/settings/notifications</code> · <code>/settings/privacy</code> · <code>/password</code>
DELETE	<code>/account</code>

`/api/mood` — Mood tracking (7)

GET	<code>/</code> <code>/streak</code> <code>/stats</code> <code>/history</code> <code>/report</code> <code>/report.pdf</code>
POST	<code>/</code>

`/api/journal` (5) · `/api/session-notes` (4) · `/api/journey` (1)

GET/POST/PUT/DELETE	journal entries · session notes · GET journey timeline
---------------------	--

`/api/appointments` — Booking (9)

GET	<code>/</code>	List
GET	<code>/:id</code>	Detail
POST	<code>/</code>	Book (enforces availability + auto-reject)
PUT	<code>/:id</code>	Update
PUT	<code>/:id/reschedule</code>	Move to a new slot
POST	<code>/:id/documents</code>	Attach intake file
GET	<code>/:id/documents/:docId</code>	Download
DELETE	<code>/:id/documents/:docId</code>	
GET	<code>/:id/details.pdf</code>	PDF summary

/api/counselors — Discovery (4)

GET /	Search, filter, sort
GET /:id	Profile
GET /:id/reviews	Ratings
GET /:id/slots	Real bookable slots from the counselor's schedule

/api/messages (7) • /api/video (4) • /api/notifications (2)

Chat threads, attachments, read receipts • Call contacts/history/stats

/api/ai — AI features (4)

POST /match	Questionnaire-based matching
GET /recommended	Profile + mood-tag based recommendation
GET /insights	Mood insights
GET /summary	Journey summary

/api/doctor — Counselor panel (57)

Dashboard, analytics, badges, notifications, feedback + reply, reports (PDF/CSV/daily), AI assistant, profile + photo, settings, password, patients (+ CSV/PDF export), journals (+ PDF), requests, accept/reject, session summarisation, appointments, availability, notes (CRUD + PDF + AI), messages, login history, documents (upload/download/share/delete).

/api/admin — Admin panel (42)

Dashboard, analytics, reports + download, audit log, users CRUD, applications (review/approve/reject/documents), counselors CRUD, appointments, sessions, calls, feedback, payments, notifications (send/read/delete), support tickets, settings, profile.

/api/emergency (5) • /api/support (5) • /api/shared-files (2) • /api/applications (1) • /api/billing (3) • /api/feedback (3)

GET /api/emergency/resources is **deliberately unauthenticated** — someone in crisis must reach helplines without logging in.

8. Functionality — User Panel

Module	Capabilities
Auth	Register, login, logout, JWT sessions, remember-me, password reset by OTP, change password
Dashboard	Greeting, upcoming sessions, mood overview, AI insight, recommended counselors, quick actions, live session counter
Discovery	Search, filter by specialty, sort (rating/experience/fee/sessions/name), counselor profile with reviews, real bookable slots
AI Matching	Questionnaire → ranked counselors with match % and reason
Appointments	Book, reschedule, cancel, online/in-person mode, reason field, document attachments, PDF details
Sessions	Video and voice calls, screen share, in-call chat, timer, call history
Chat	Real-time messaging, attachments, voice notes, typing indicator, read receipts
Mood	Daily entry (1–5), intensity (1–10), context tags, notes, streak, history, weekly/monthly report + PDF
Journal	Entries with mood, tags, private/shared toggle
AI Summary	Journey timeline, before/after growth by life area, data-derived recommendations
Payments	Checkout (simulated), billing history, receipts, automatic refunds
Feedback	Rate and review a completed session; see the counselor's reply
My Files	Files the counselor shared, with preview and download
Emergency	Region-aware crisis helplines (works logged out), grounding exercises, emergency contact
Support	Searchable FAQ, support tickets with admin replies

9. Functionality — Counselor Panel

Module	Capabilities
Workflow	Client books → pending request → accept/reject with reason → notifications both sides → payment gated on acceptance → auto-refund on cancel
Dashboard	Today's sessions, upcoming, completed, monthly stats, pending requests, total patients
Profile	Specializations, qualifications, licence, experience, location, fees, languages, photo upload
Appointments	Request queue, accept/decline, reschedule, calendar view, PDF summary, new-appointment modal
Patients	Profiles, history, mood journals, notes, progress, CSV + branded PDF export , risk filter
Sessions	Video/voice calling, screen share, in-call chat, timer, history
Notes	Create/edit/delete, tags, patient link, session anchoring, private/shared, PDF export, AI summarise, AI draft, voice dictation
AI Assistant	Intent engine answering from real records — patients, schedule, risk, mood trends, ratings, revenue, techniques
Reports	Monthly, patient stats, revenue, daily breakdown, PDF/Excel export, period filter
Analytics	Caseload mood over time, patient concerns, retention, peak hours, day-of-week distribution
Availability	Weekly schedule with add/remove slots, vacation mode , break times, auto-reject of out-of-hours requests
Documents	Upload/download/preview/delete; share with a specific patient
Feedback	View reviews and reply (reply reaches the client + notification)
Security	Real TOTP 2FA — QR enrolment, recovery codes, login challenge, password-confirmed disable; login history

10. Functionality — Admin Panel

Module	Capabilities
Dashboard	Live counts, stat cards, user growth, weekly appointments, session categories, recent activity, top counselors, system status
Users	List, create, view, edit, delete, suspend
Counselors	Full CRUD, verification status
Applications	Review counselor applications, view uploaded credentials, approve → creates account, or reject
Appointments / Sessions	Platform-wide monitoring, call log
Payments	Transactions, revenue, counselor payouts (80%), platform fee (20%), refunds, 6-month trend, CSV export
Analytics / Reports	Platform metrics, downloadable reports
Notifications	Broadcast to Everyone / Clients / Counselors / Admins , real scheduling, delete retracts
Support	Ticket queue with reply and status management
Settings / Profile	Platform config, admin profile, password

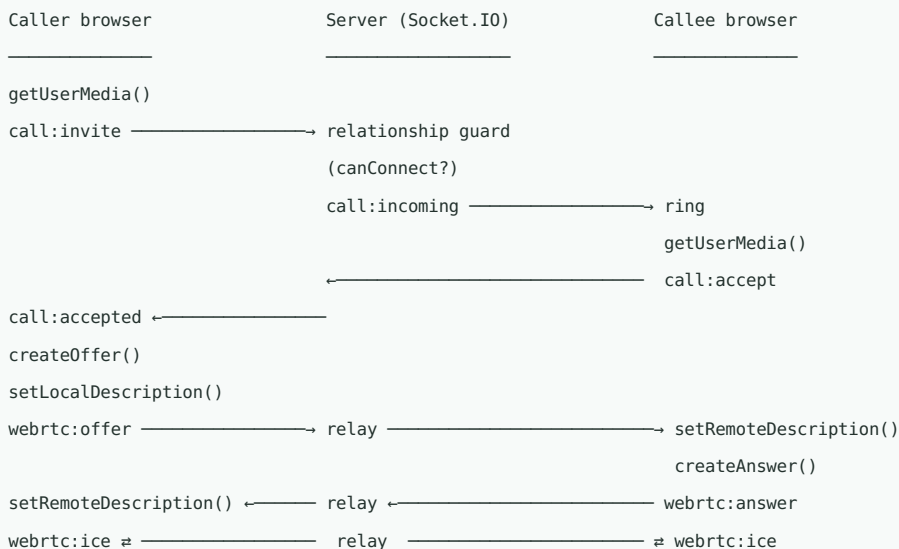
Deliberately absent: admins cannot read private counseling chat threads. Reading confidential therapeutic conversations is a confidentiality violation, so the feature was removed. **This is a strong viva point.**

11. Key Data Flows

11.1 Booking → payment → session

1. Client picks a slot	GET /counselors/:id/slots (generated from the counselor's saved schedule, minus booked slots, past times, break windows and vacation dates)
2. Client books	POST /appointments → status: pending availability.checkBookable() enforces hours; auto-reject declines out-of-hours immediately
3. Counselor is notified	Socket + notification feed
4. Counselor accepts	PUT /doctor/appointments/:id/accept → confirmed
5. Client pays	POST /billing/pay → payments record, paymentStatus: paid
6. Session runs	WebRTC video call
7. Counselor completes	status: completed
8. Client reviews	POST /feedback → counselor rating recomputed
9. Counselor replies	POST /doctor/feedback/:id/reply → client notified

11.2 Video call establishment



|| MEDIA FLOWS DIRECTLY BROWSER ↔ BROWSER (WebRTC / STUN) ||
 || Audio and video NEVER pass through the server ||

11.3 Counselor onboarding

```
Public form (Join as Counselor)
  → multipart upload of credentials (stored outside the public folder)
  → POST /applications → status: pending
  → Admin reviews at /admin/applications, views documents
  → Approve → creates a doctors record with a hashed temporary password
    → counselor appears in client-facing discovery
  → Reject → recorded with reason
```

11.4 Authentication with 2FA

```
POST /auth/login { email, password }
  └─ bcrypt.compare(password, passwordHash)    12 salt rounds
  └─ 2FA disabled → sign JWT { id, role }      → full access
  └─ 2FA enabled  → sign JWT { id, role, stage:'2fa' } → CHALLENGE ONLY
      |
      |   authenticate middleware REJECTS stage:'2fa' tokens.
      |   A password alone must never open the API.
      ▼
POST /auth/2fa/verify { code }
  └─ TOTP verify (±1 step, timingSafeEqual) → full JWT
```

12. Security Model

Layer	Implementation
Password storage	bcrypt, 12 salt rounds , per-password salt
Session	JWT (HS256), 7-day expiry, in-memory blacklist on logout
Two-factor	TOTP to RFC 6238 — HMAC-SHA1, 30s step, ± 1 window, constant-time compare, recovery codes
Authorization	<code>requireRole()</code> on every protected route
Relationship guard	<code>relationship.service.canConnect()</code> — a counselor may only reach patients they actually work with; enforced on chat, calls, journals and documents
2FA token isolation	Challenge tokens carry <code>stage: '2fa'</code> and are rejected by the API middleware
Input validation	express-validator on every mutating route
HTTP headers	Helmet
CORS	Restricted to the configured frontend origin
File uploads	Multer with type allow-list and 25 MB cap; clinical documents stored outside the statically-served folder
Password reset	Generic response (no account enumeration), 10-min expiry, 5 attempts, 60s resend cooldown
Confidentiality	Admin access to counseling chat deliberately removed

13. Known Limitations

State these before an examiner finds them.

#	Limitation	Detail
1	<code>/ai/match</code> <code>match %</code> is not deterministic	<code>computeMatch()</code> seeds the score with <code>Math.random()*20+75</code> and adds +5/+8 for keyword hits. See the warning in <code>TECH-STACK-AND-ALGORITHMS.md</code> §4.1 — fix before viva
2	Payments are simulated	No real gateway. <code>simulated: true</code> on every record
3	No email/SMS delivery	Reset codes print to the server console; there is no mailer
4	Single-process database assumption	Reads are served from an in-process cache; correct for one backend instance
5	No automated test suite	Testing was manual plus scripted integration checks, not committed unit tests
6	Not deployed	Runs locally; deployment plan documented
7	JWT in <code>localStorage</code>	Vulnerable to XSS; <code>httpOnly</code> cookies + CSRF would be the production choice
8	No TURN server	Calls may fail behind symmetric NAT; STUN only
9	No TypeScript type-checking step	No <code>tsconfig.json</code> , so TS types are documentation rather than enforcement
10	Email verification absent	Accounts are usable immediately

Generated from the CounselConnect codebase. Every file, endpoint, collection and field listed above was verified against the source at the time of writing.